In each of these cases, you can provide a content type for the data, and a filename to be used by the client. Browsers offer to save binary data files using the suggested filename. The `File` and `Byte` methods even support range requests by specifying `enableRangeProcessing` as `true`.

DEFINITION Clients can create range requests using the `Range` header to request a specific range of bytes from the server instead of the whole file, reducing the bandwidth required for a request. When range requests are enabled for `Results.File()` or `Results.Byte()`, ASP.NET Core automatically handles generating an appropriate response. You can read more about range requests at http://mng.bz/Wzd0.

If the built-in `Results` helpers don't provide the functionality you need, you can always fall back to creating a response manually, as in listing 5.4. If you find yourself creating the same manual response several times, you could consider creating a custom `IResult` type to encapsulate this logic. I show how to create a custom `IResult` that returns XML and registers it as an extension in this blog post: http://mng.bz/8rNP.

5.4 Running common code with endpoint filters

In section 5.3 you learned how to use `Results` to return different responses when the request isn't valid. We'll look at validation in more detail in chapter 7, but in this section, you'll learn how to use filters to extract common code that executes before (or after) an endpoint executes.

Let's start by adding some extra validation to the `fruit` API from listing 5.5. The following listing adds an additional check to the `MapGet` endpoint